Tugas Kecil II IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Voxelization Objek 3D menggunakan Octree

Ariel Cornelius Sitorus 13524085
Manuel Thimoty Silalahi 13524102

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

Contents

BAB II..... 4
ALGORITMA BRUTE FORCE..... 4
BAB III SOURCE CODE..... 6
REFERENSI..... 20
LAMPIRAN..... 21
Tabel Hasil..... 21
Pranala Github..... 21
Pernyataan..... 21

BAB I
DESKRIPSI PERMASALAHAN

Voxelization merupakan sebuah proses konversi objek tiga dimensi dari format *polygon mesh* menjadi model yang tersusun atas kumpulan kubus kecil yang disebut sebagai voxel. Analogi paling mudahnya adalah mengubah model halus menjadi bentuk kotak-kotak mirip seperti di dalam permainan *Minecraft*.

Dalam tugas ini, masalah utama yang harus diselesaikan adalah membangun program yang mampu mengubah file ".obj" menjadi representasi voxel secara efisien. Tantangannya adalah bagaimana kita bisa merepresentasikan ruang 3D tanpa memakan banyak memori, mengingat grid 3D biasa akan memproses area kosong secara mubazir. Oleh karena itu, solusi yang diterapkan adalah menggunakan struktur data Octree sebuah pohon hierarkis di mana setiap percabangannya membagi ruang menjadi 8 anak kubus yang lebih kecil.

BAB II

ALGORITMA

DIVIDE AND CONQUER

Algoritma divide and conquer adalah strategi penyelesaian masalah yang membagi satu persoalan besar menjadi beberapa sub-persoalan yang lebih kecil dengan tipe yang sama, menyelesaikannya secara mandiri, lalu menggabungkan hasilnya kembali. Pada dasarnya, algoritma ini bekerja dengan tiga prinsip utama: membagi (*divide*), menyelesaikan secara rekursif (*conquer*), dan menggabungkan solusi (*combine*).

Pada persoalan kali ini, divide and conquer-nya bekerja dengan cara membagi ruang 3D (kubus) menjadi delapan sub-ruang yang lebih kecil secara terus-menerus menggunakan struktur data *Octree*. Alih-alih memeriksa seluruh ruang secara sekaligus, algoritma ini hanya fokus pada bagian ruang yang bersinggungan dengan permukaan model (segitiga). Jika suatu bagian ruang tidak berisi segitiga, maka bagian tersebut langsung dibuang (*pruning*), sehingga komputasi menjadi jauh lebih efisien dan cepat.

Langkah-langkah algoritma divide and conquer yang dibuat untuk vokselisasi ini adalah sebagai berikut:

1. **Inisialisasi Root**: Program mencari nilai koordinat minimum dan maksimum dari seluruh titik objek pada file .obj. Dari nilai tersebut, dibuat satu kotak pembungkus awal (*bounding box*) berbentuk kubus yang memuat seluruh objek sebagai titik awal rekursi.
2. **Pembagian Ruang (Divide)**: Setiap kotak yang sedang diproses dibagi menjadi 8 anak kotak (*octants*). Penentuan posisi pusat masing-masing anak kotak dilakukan dengan menggeser titik pusat induknya ke arah delapan penjuru (atas-bawah, kiri-kanan, depan-belakang) menggunakan operasi bitwise.
3. **Filter Segitiga (Conquer)**: Untuk setiap anak kotak, program melakukan pengecekan apakah ada segitiga yang bersinggungan dengan volume kotak tersebut menggunakan *Separating Axis Theorem* (SAT) yang menguji 13 sumbu proyeksi.
    - Jika kotak kosong (tidak ada segitiga), proses pada cabang tersebut dihentikan.
    - Jika kotak berisi segitiga, daftar segitiga tersebut diteruskan untuk dibagi lagi pada tingkat kedalaman berikutnya.
4. **Penentuan Voxel (Base Case)**: Proses pembagian berhenti jika kedalaman rekursi sudah

mencapai batas maxDepth yang ditentukan. Kotak pada level terdalam yang masih berisi segitiga inilah yang ditetapkan sebagai **Voxel**.

5. **Penyusunan Output (Combine)**: Seluruh voxel yang ditemukan dikumpulkan, lalu masing-masing voxel dikonversi menjadi data jaring (*mesh*) kubus standar (terdiri dari 8 titik dan 12 segitiga) untuk kemudian dituliskan ke dalam file .obj sebagai hasil akhir.

BAB III
SOURCE
CODE

Alur Program Secara Keseluruhan

1. main.cpp : file inti yang berisi implementasi dari kelas Octree, pengecekan AABB, dan fungsi utama (main)

Definisi Struct

```
// struct untuk point
struct VectorV{
    float x,y,z;
};
// face untuk simpan indeks dari point
struct Face{
    int i,j,k;
};
struct AABB{
    VectorV minPoint; VectorV maxPoint;
    bool intersect(const AABB& other) const {
        return (minPoint.x <= other.maxPoint.x && maxPoint.x >=
other.minPoint.x
        && minPoint.y <= other.maxPoint.y && maxPoint.y >= other.minPoint.y
        && minPoint.z <= other.maxPoint.z && maxPoint.z >= other.minPoint.z);
    }
};
```

Octree

```
class Octree{
    public :
        AABB bound;// untuk bound
        bool isLeaf;// bool apakah ini daun
        int curDepth;// kedalaman sekarang
        vector<int> faceIds; // faceId yg disimpan di node ini
        Octree* children[8]; // ke8 anaknya

        // konstruktor
        Octree(AABB b, int d){
            bound = b;
            curDepth = d;
            isLeaf = true;
            for(int i = 0 ; i < 8 ; i++) children[i] = nullptr;
            if(d > 0) statTerbentuk[d]++;
        }

        void subdivide(){
            float x_mid = (bound.minPoint.x + bound.maxPoint.x) / 2.0f;
            float y_mid = (bound.minPoint.y + bound.maxPoint.y) / 2.0f;
            float z_mid = (bound.minPoint.z + bound.maxPoint.z) / 2.0f;
            children[0] = new Octree({{bound.minPoint.x, bound.minPoint.y,
bound.minPoint.z}, {x_mid, y_mid, z_mid}}, curDepth + 1);
            children[1] = new Octree({{x_mid, bound.minPoint.y,
bound.minPoint.z}, {bound.maxPoint.x, y_mid, z_mid}}, curDepth + 1);
            children[2] = new Octree({{bound.minPoint.x, bound.minPoint.y,
z_mid}, {x_mid, y_mid, bound.maxPoint.z}}, curDepth + 1);
            children[3] = new Octree({{x_mid, bound.minPoint.y, z_mid},
```

```
{bound.maxPoint.x, y_mid, bound.maxPoint.z}}, curDepth + 1);

            // Atas (Y mid ke Y maxPoint)
            children[4] = new Octree({{bound.minPoint.x, y_mid,
bound.minPoint.z}, {x_mid, bound.maxPoint.y, z_mid}}, curDepth + 1);
            children[5] = new Octree({{x_mid, y_mid, bound.minPoint.z},
{bound.maxPoint.x, bound.maxPoint.y, z_mid}}, curDepth + 1);
            children[6] = new Octree({{bound.minPoint.x, y_mid, z_mid},
{x_mid, bound.maxPoint.y, bound.maxPoint.z}}, curDepth + 1);
            children[7] = new Octree({{x_mid, y_mid, z_mid},
{bound.maxPoint.x, bound.maxPoint.y, bound.maxPoint.z}}, curDepth + 1);

            isLeaf = false;
        }

        void buildOctree(const vector<int>& inputFaceId, const vector<Face>&
globalFaces, const vector<VectorV>& globalVertices, int maxDepth){
            if(curDepth >= maxDepth){
                faceIds = inputFaceId;
                return;
            }

            subdivide();

            for(int faceId : inputFaceId){
                VectorV v0 = globalVertices[globalFaces[faceId].i];
                VectorV v1 = globalVertices[globalFaces[faceId].j];
                VectorV v2 = globalVertices[globalFaces[faceId].k];

                for(int i = 0 ; i < 8 ; i++){
                    // yang sebelumnya AABB jadi pengecekan triangle
                    if(checkTriangleBox(children[i]->bound.minPoint,
children[i]->bound.maxPoint, v0, v1, v2)) {
                        children[i]->faceIds.push_back(faceId);
                    }
                }
            }

            for(int i = 0  ; i < 8 ; i++){
                if(!children[i]->faceIds.empty()){

children[i]->buildOctree(children[i]->faceIds,globalFaces,globalVertices,maxDe
pth);
                }
                else{
                    int anakDepth = children[i]->curDepth;
                    if(anakDepth <= maxDepth)
statTidakDitelusuri[anakDepth]++;
                }
            }

            faceIds.clear();
        }

};
```

AABB

```
struct AABB{
    VectorV minPoint; VectorV maxPoint;
    bool intersect(const AABB& other) const {
        return (minPoint.x <= other.maxPoint.x && maxPoint.x >=
other.minPoint.x
        && minPoint.y <= other.maxPoint.y && maxPoint.y >= other.minPoint.y
        && minPoint.z <= other.maxPoint.z && maxPoint.z >= other.minPoint.z);
    }
};
```

```
bool checkTriangleBox(const VectorV& boxMin, const VectorV& boxMax, const
VectorV& v0, const VectorV& v1, const VectorV& v2) {
    // 1. Cek Cepat (AABB vs AABB)
    float triMinX = min({v0.x, v1.x, v2.x}), triMaxX = max({v0.x, v1.x,
v2.x});
    float triMinY = min({v0.y, v1.y, v2.y}), triMaxY = max({v0.y, v1.y,
v2.y});
    float triMinZ = min({v0.z, v1.z, v2.z}), triMaxZ = max({v0.z, v1.z,
v2.z});

    if (triMaxX < boxMin.x || triMinX > boxMax.x) return false;
    if (triMaxY < boxMin.y || triMinY > boxMax.y) return false;
    if (triMaxZ < boxMin.z || triMinZ > boxMax.z) return false;

    // Cek Bidang Segitiga
    // Hitung normal segitiga
    VectorV edge0 = subVec(v1, v0);
    VectorV edge1 = subVec(v2, v1);
    VectorV normal = crossProduct(edge0, edge1);

    // Cari titik di kotak yang paling jauh searah normal dan berlawanan arah
normal
    VectorV vmin, vmax;
    if(normal.x > 0) {vmin.x = boxMin.x; vmax.x = boxMax.x; } else { vmin.x =
boxMax.x; vmax.x = boxMin.x; }
    if(normal.y > 0) {vmin.y = boxMin.y; vmax.y = boxMax.y; } else { vmin.y =
boxMax.y; vmax.y = boxMin.y; }
    if(normal.z > 0) {vmin.z = boxMin.z; vmax.z = boxMax.z; } else { vmin.z =
boxMax.z; vmax.z = boxMin.z; }

    // Jika seluruh kotak ada di luar bidang segitiga
    if(dotProduct(normal, subVec(vmin, v0)) > 0) return false;
    if(dotProduct(normal, subVec(vmax, v0)) < 0) return false;

    return true;
}
```

buildResult

```
void buildResult(Octree* node){
    if(!node->isLeaf){
        for(int i = 0 ; i < 8 ; i++){
            if(node->children[i] != nullptr){
                buildResult(node->children[i]);
            }
        }
    }
    else{
        finalDepth = max(finalDepth,node->curDepth);
        if (node->faceIds.empty()) return;

        VectorV minP = node->bound.minPoint;
        VectorV maxP = node->bound.maxPoint;

        int curIndex = voxelVertices.size();

        // 8 titik kubus
        voxelVertices.push_back({minP.x, minP.y, minP.z}); // 0: Kiri Bawah
Depan
        voxelVertices.push_back({maxP.x, minP.y, minP.z}); // 1: Kanan Bawah
```

```
Depan
        voxelVertices.push_back({maxP.x, minP.y, maxP.z}); // 2: Kanan Bawah
Belakang
        voxelVertices.push_back({minP.x, minP.y, maxP.z}); // 3: Kiri Bawah
Belakang

        // atas
        voxelVertices.push_back({minP.x, maxP.y, minP.z}); // 4: Kiri Atas
Depan
        voxelVertices.push_back({maxP.x, maxP.y, minP.z}); // 5: Kanan Atas
Depan
        voxelVertices.push_back({maxP.x, maxP.y, maxP.z}); // 6: Kanan Atas
Belakang
        voxelVertices.push_back({minP.x, maxP.y, maxP.z}); // 7: Kiri Atas
Belakang

        // buat 12 segitiga
        // Sisi Bawah
        voxelFaces.push_back({curIndex + 0, curIndex + 2, curIndex + 1});
        voxelFaces.push_back({curIndex + 0, curIndex + 3, curIndex + 2});
        // Sisi Atas
        voxelFaces.push_back({curIndex + 4, curIndex + 5, curIndex + 6});
        voxelFaces.push_back({curIndex + 4, curIndex + 6, curIndex + 7});
        // Sisi Depan
        voxelFaces.push_back({curIndex + 0, curIndex + 1, curIndex + 5});
        voxelFaces.push_back({curIndex + 0, curIndex + 5, curIndex + 4});
        // Sisi Belakang
        voxelFaces.push_back({curIndex + 3, curIndex + 6, curIndex + 2});
        voxelFaces.push_back({curIndex + 3, curIndex + 7, curIndex + 6});
        // Sisi Kiri
        voxelFaces.push_back({curIndex + 0, curIndex + 4, curIndex + 7});
        voxelFaces.push_back({curIndex + 0, curIndex + 7, curIndex + 3});
        // Sisi Kanan
        voxelFaces.push_back({curIndex + 1, curIndex + 2, curIndex + 6});
        voxelFaces.push_back({curIndex + 1, curIndex + 6, curIndex + 5});


    }

}
```

main

```
int main(){
    string fileName;
    cin >> fileName;
    // string fileName = "teapot";

    string path = "../test/" + fileName + ".obj";
    if(loadObject(path, vertices, faces)){
        cout << "SUKSES MEMBACA FILE!" << endl;
        AABB rootBound;
        rootBound.minPoint = {99999.0f, 99999.0f, 99999.0f};
        rootBound.maxPoint = {-99999.0f, -99999.0f, -99999.0f};

        for(auto v : vertices) {
            rootBound.minPoint.x = min(rootBound.minPoint.x, v.x);
            rootBound.minPoint.y = min(rootBound.minPoint.y, v.y);
            rootBound.minPoint.z = min(rootBound.minPoint.z, v.z);
            rootBound.maxPoint.x = max(rootBound.maxPoint.x, v.x);
            rootBound.maxPoint.y = max(rootBound.maxPoint.y, v.y);
```

```
            rootBound.maxPoint.z = max(rootBound.maxPoint.z, v.z);
        }
        vector<int> allFaceIds;
        for(int i = 0; i < faces.size(); i++) allFaceIds.push_back(i);
        int maxDepth ;
        // maxDepth = 8;
        cin >> maxDepth;
        Octree* root = new Octree(rootBound, 0);
        root->buildOctree(allFaceIds, faces, vertices, maxDepth);

        buildResult(root);
        string targetOutput = fileName + "_" + to_string(maxDepth)
+"_voxel.obj";
        ofstream outFile(targetOutput);
        if(outFile.is_open()) {
            for(auto v : voxelVertices){
                outFile << "v " << v.x << " " << v.y << " " << v.z << "\n";
            }
            for(auto f : voxelFaces){
                // .obj dimulai dari 1, jadi kita tambah 1
                outFile << "f " << (f.i + 1) << " " << (f.j + 1) << " " <<
(f.k + 1) << "\n";
            }
            outFile.close();
            cout << "berhasil! disimpan di " << targetOutput ;
        } else {
            cout << "Gagal buat output" << endl;
        }
    }
    return 0;
}
```

BAB IV
UJI COBA PROGRAM

Program dikompilasi dengan metode biasa dalam mengkompilasi file cpp menjadi executable yang sesuai. File executable diletakkan pada folder `bin/` . Untuk mengkompilasi dan menjalankan program, di root project, jalankan perintah berikut

Kompilasi
`g++ src/main.cpp -o bin/main`
Run
`./bin/main`

Program akan menerima beberapa input, yaitu :

- Path input file.obj.
- Kedalam Octree yang ingin dibentuk.
- Nama file output. (Catatan : file output akan otomatis diletakkan di folder `test`.

Lalu, program akan mengeluarkan beberapa output, yaitu

- Banyak voxel yang terbentuk.
- Banyak vertex yang terbentuk.
- Banyak faces yang terbentuk.
- Statistik node yang terbentuk.
- Statistik node yang tidak perlu ditelusuri.
- Kedalam Octree
- Waktu proses.
- Path file output.

Berikut adalah garis besar program :

Program

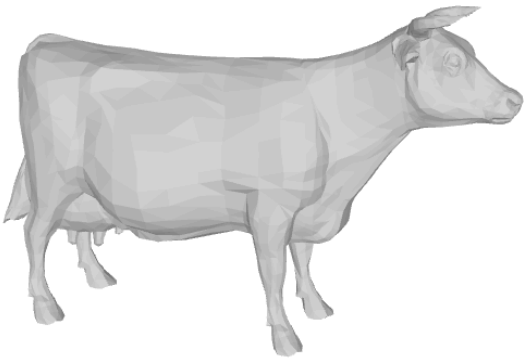
```
Masukkan nama file input (dari test/input/, format .obj) : <path file input>

SUKSES MEMBACA FILE!

Masukkan kedalam maksimum dari Octree : <kedalaman maksimum>

Banyak voxel yang terbentuk : <banyak voxel yang terbentuk>
Banyak vertex yang terbentuk : <banyak vertex yang terbentuk>
Banyak faces yang terbentuk : <banyak faces yang terbentuk>
```

```
Statistik node octree yang terbentuk :
0 : <banyak node yang terbentuk dengan depth 1>
1 : <banyak node yang terbentuk dengan depth 2>
.
.
.
n : <banyak node yang terbentuk dengan depth n>


Statistik node octree yang tidak perlu ditelusuri :
0 : <banyak node dengan depth 0 yang tidak perlu ditelusuri>
1 : <banyak node dengan depth 1 yang tidak perlu ditelusuri>
.
.
.
n :  <banyak node dengan depth n yang tidak perlu ditelusuri>

Kedalaman Octree : <kedalaman octree>

Durasi Proses : <waktu proses dalam satuan ms>

Masukkan nama file output : <nama output file (catatan : akan terotomatis
tersave dengan format <namafile>_voxel.obj>)
Berhasil! disimpan di <path output file>

Mau voxel lagi? (y/n) :
```

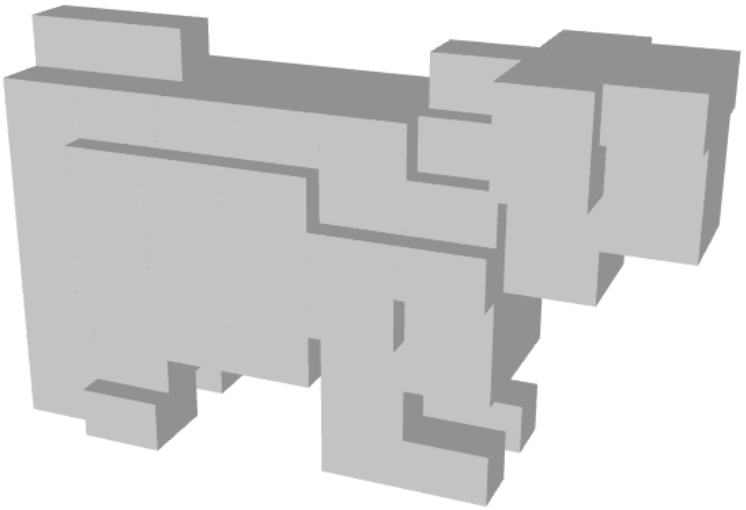
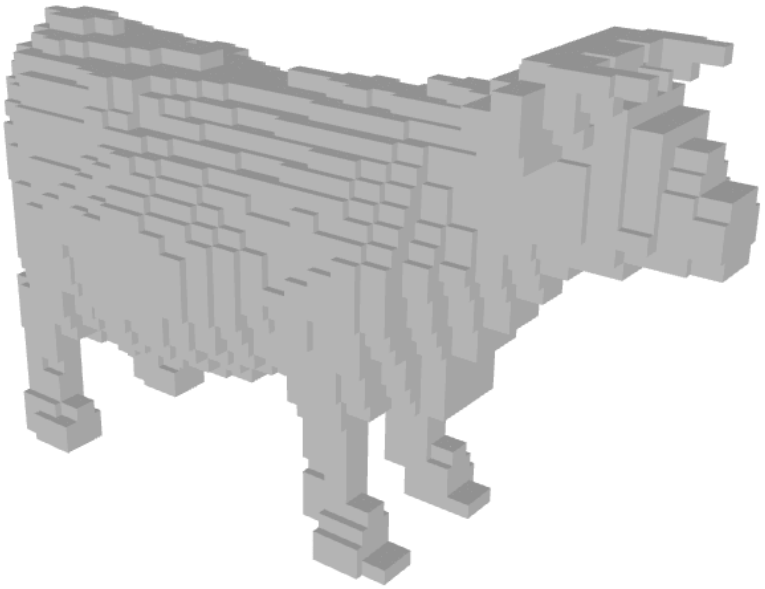
Kita akan menggunakan 3 file .obj berbeda dalam pengujian, setiap file yang diuji akan diuji dengan kedalaman maksimum yang berbeda - beda sebanyak 3 juga. Berikut adalah Tabel Hasil pengujiannya.

1. Cow.obj

   Berikut adalah gambar originalnya :



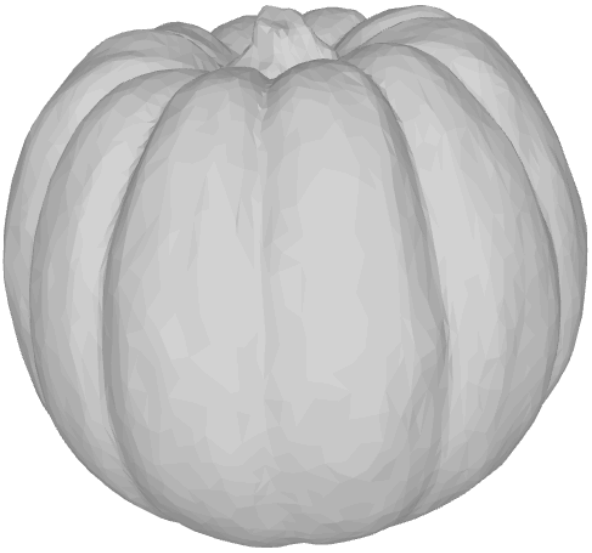
Kedalaman	Gambar

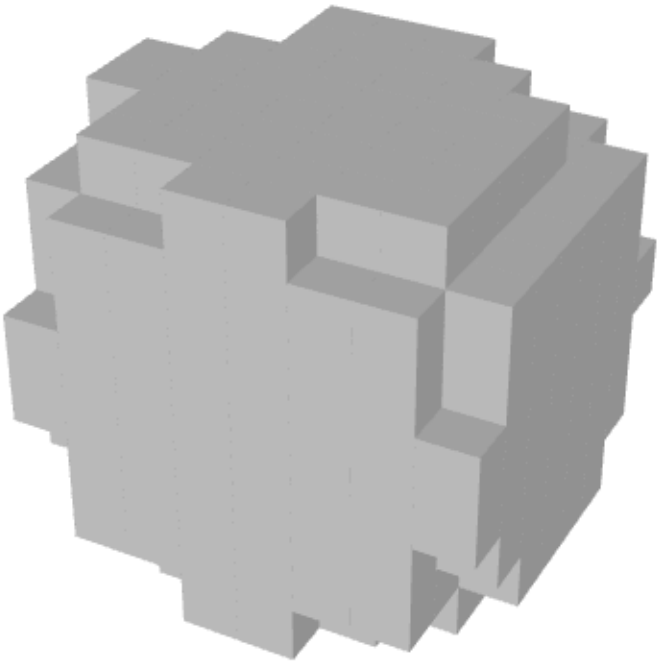
3	
5	

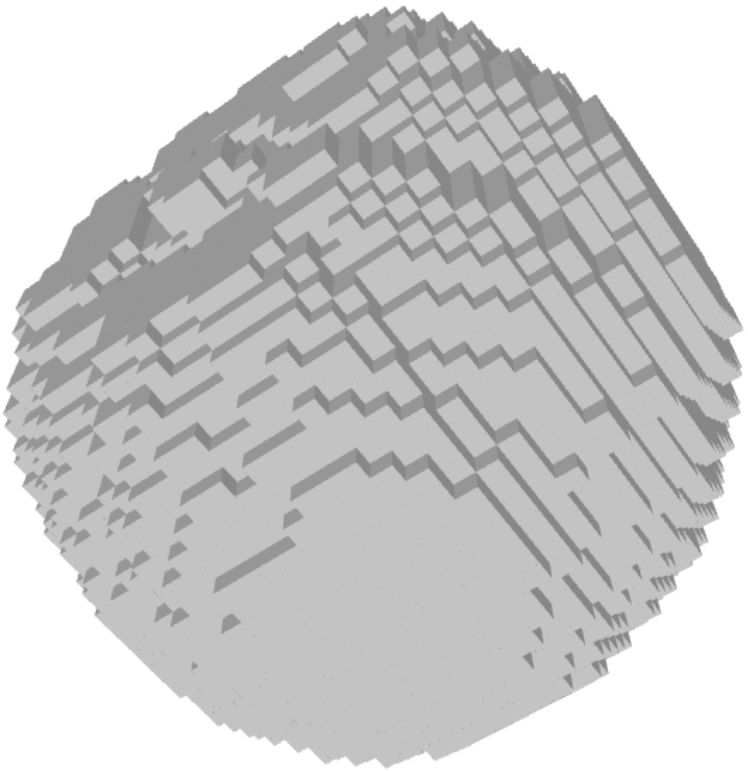
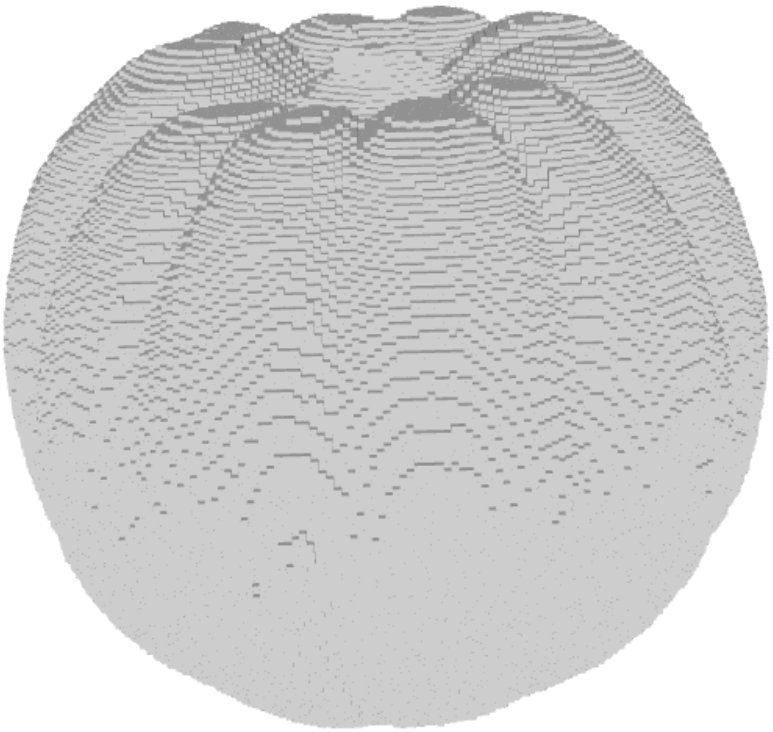
7	

2. pumpkin.obj

   Berikut adalah gambar originalnya :

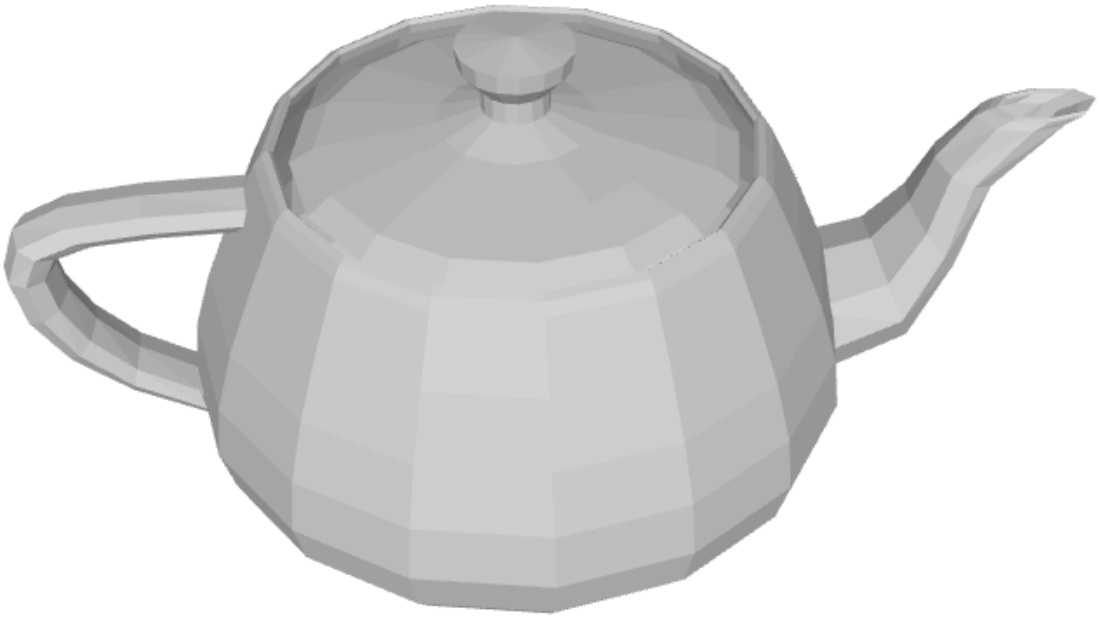


Kedalaman	Gambar
3	

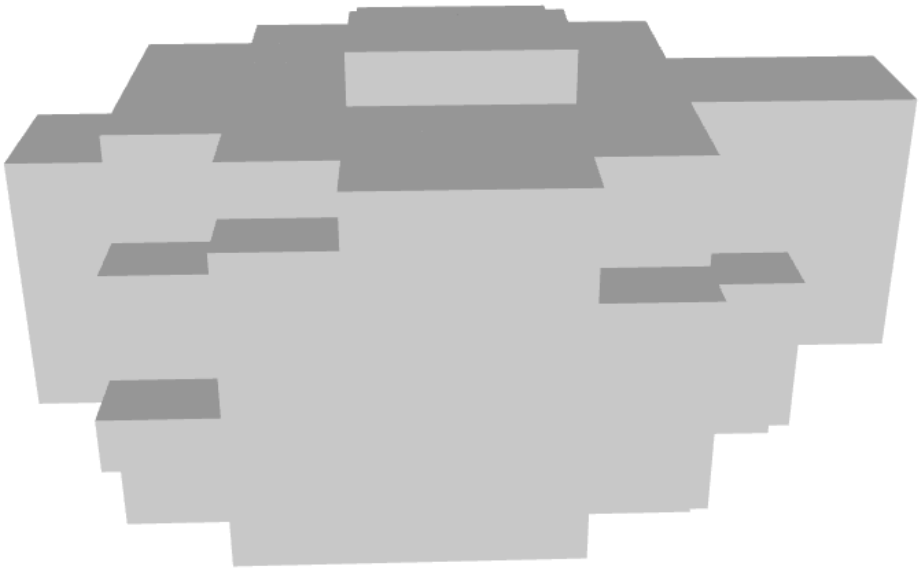
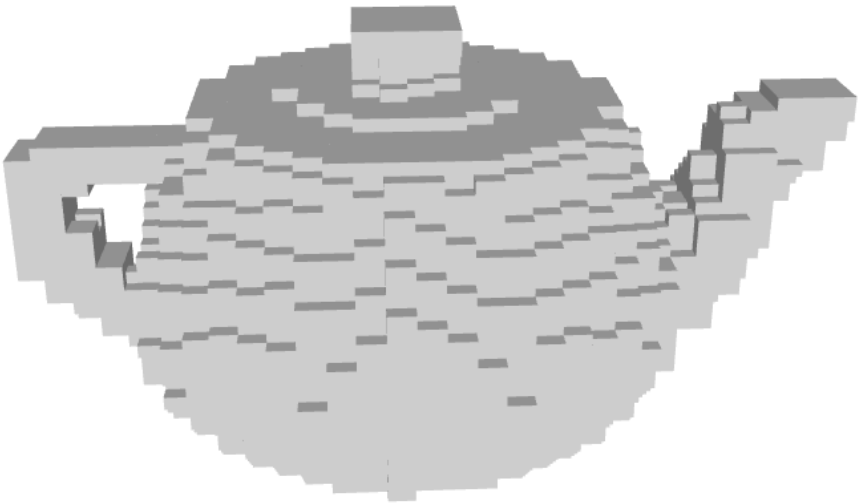
5	
7	

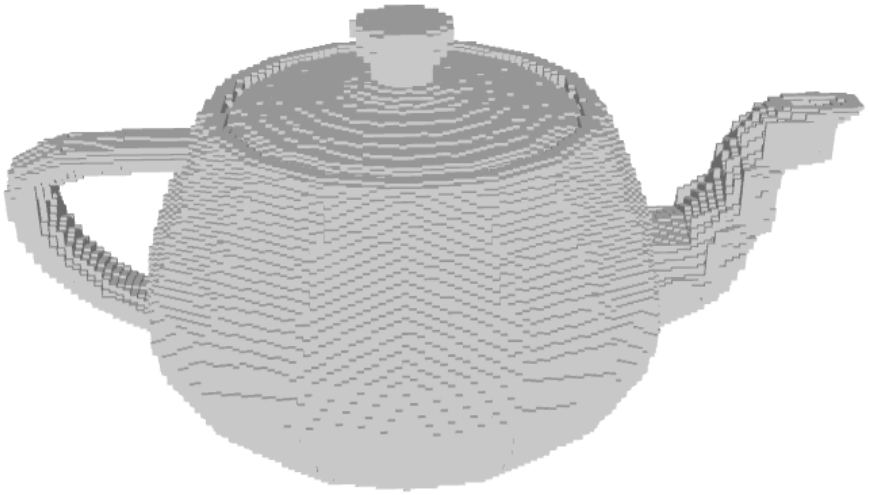
3. teapot.obj

   Berikut adalah gambar originalnya :



Kedalaman	Gambar

3	
5	

7	

REFERENSI

- R. Munir, “Algoritma Divide and Conquer (Bagian 1),” IF2211 Strategi Algoritma, 2026. [Online]. Tersedia: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/07-Algoritma-Divide-and-Conquer-(2026)-Bagian1.pdf
- R. Munir, “Algoritma Divide and Conquer (Bagian 2),” IF2211 Strategi Algoritma, 2026. [Online]. Tersedia: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/08-Algoritma-Divide-and-Conquer-(2026)-Bagian2.pdf
- R. Munir, “Algoritma Divide and Conquer (Bagian 3),” IF2211 Strategi Algoritma, 2026. [Online]. Tersedia: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/09-Algoritma-Divide-and-Conquer-(2026)-Bagian3.pdf
- R. Munir, “Algoritma Divide and Conquer (Bagian 4),” IF2211 Strategi Algoritma, 2026. [Online]. Tersedia: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/10-Algoritma-Divide-and-Conquer-(2026)-Bagian4.pdf
- J. Hu, X. Gong, J. Zhang, J. Geng, dan M. Wu, "An Efficient Path Planning Algorithm for Unmanned Aerial Vehicles Based on Octree Maps and Improved A* Algorithm," *Sensors*, vol. 22, no. 18, p. 7040, 2022. [Online]. Tersedia: https://www.mdpi.com/1424-8220/22/18/7040
- J. Doe (et al.), "Octants, Octree Structure, and Linear Octree with Locational Codes," *ResearchGate*, 2022. [Online]. Tersedia: https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

LAMPIRAN

Tabel Hasil

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	YA	
2	Program berhasil dijalankan	YA	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	YA	
4	Program informasi-informasi atau report	YA	
5	Program dapat menerima masukan .obj, dan mengembalikan output	YA	
6	[Bonus] Program menggunakan *concurrency*	YA	
7	[Bonus] Program dapat menampilkan visualisasi .obj		TIDAK

Pranala Github

Repository Github dapat diakses melalui pranala berikut:

https://github.com/manuellthimoty/Tucil2_13524102_13524085.git

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri

Manuel Thimoty Silalahi

Ariel Cornelius Sitorus